

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 TOTAL MARKS: 50

Annexure A

Coding Challenge

Code of Conduct:

*I **T Sanhith(192311228)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Coding Challenge

1. Write a program to simulate the IPsec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

Aim:

To simulate the IPsec Authentication Header process by generating and verifying a Message Authentication Code (MAC) to ensure the integrity and authenticity of an IP packet.

```
import hmac
import hashlib

# Shared secret key
secret_key = b"IPSecSecretKey"

# Original IP packet
ip_packet = "Source: 192.168.1.10 | Destination: 192.168.1.20 | Data: Hello"

# Generate MAC using HMAC-SHA256
mac = hmac.new(
    secret_key,
    ip_packet.encode(),
    hashlib.sha256
).hexdigest()

print("Original IP Packet:")
print(ip_packet)

print("\nGenerated Authentication Header MAC:")
print(mac)

# Receiver verifies the MAC
received_packet = ip_packet
received_mac = mac

calculated_mac = hmac.new(
    secret_key,
    received_packet.encode(),
    hashlib.sha256
).hexdigest()

if hmac.compare_digest(received_mac, calculated_mac):
    print("\nPacket Integrity Verified")
    print("Authentication Successful")
else:
    print("\nPacket Modified")
    print("Authentication Failed")
```

Console

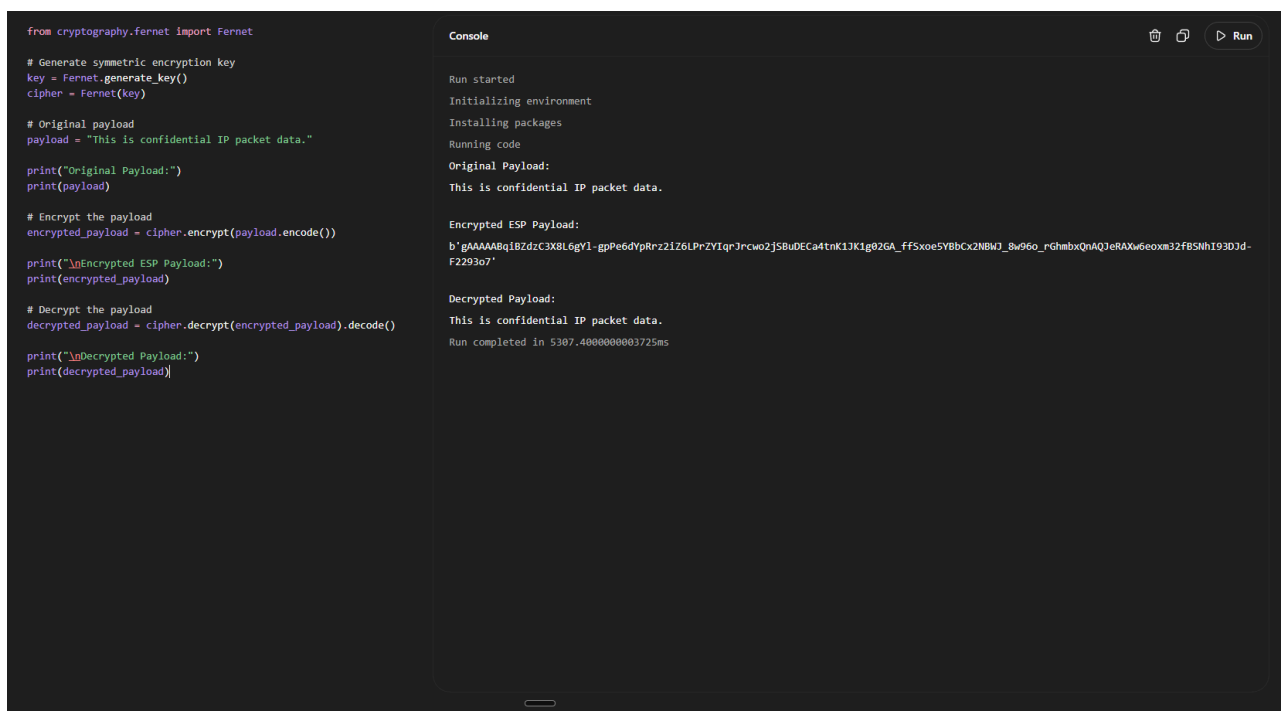
Run started
Initializing environment
Installing packages
Running code
Original IP Packet:
Source: 192.168.1.10 | Destination: 192.168.1.20 | Data: Hello
Generated Authentication Header MAC:
477b6233df0dd832210d945fe6d0a93cbc13924a54774c8f7c67e7e78916ef6
Packet Integrity Verified
Authentication Successful
Run completed in 5581.199999999255ms

Result: Thus, the IPsec Authentication Header process was simulated successfully using HMAC-SHA256 to verify packet integrity and authentication.

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

Aim:

To implement a simplified IPsec ESP mechanism for encrypting and decrypting an IP packet payload.



```
from cryptography.fernet import Fernet

# Generate symmetric encryption key
key = Fernet.generate_key()
cipher = Fernet(key)

# Original payload
payload = "This is confidential IP packet data."

print("Original Payload:")
print(payload)

# Encrypt the payload
encrypted_payload = cipher.encrypt(payload.encode())

print("\nEncrypted ESP Payload:")
print(encrypted_payload)

# Decrypt the payload
decrypted_payload = cipher.decrypt(encrypted_payload).decode()

print("\nDecrypted Payload:")
print(decrypted_payload)
```

Console

```
Run started
Initializing environment
Installing packages
Running code
Original Payload:
This is confidential IP packet data.

Encrypted ESP Payload:
b'gAAAAABqIBZdzC3X8L6gVl-gpPe6dYpRr2ZiZ6LPrZYIq3Jrcwo2jS8uDECa4tnK1JKI802GA_ff5xoe5VBbCx2NBWJ_8u96o_rGhmbxQnAQJ6RAXw6eoxm32f85NhI9303d-F2293o7'

Decrypted Payload:
This is confidential IP packet data.
Run completed in 5307.4000000003725ms
```

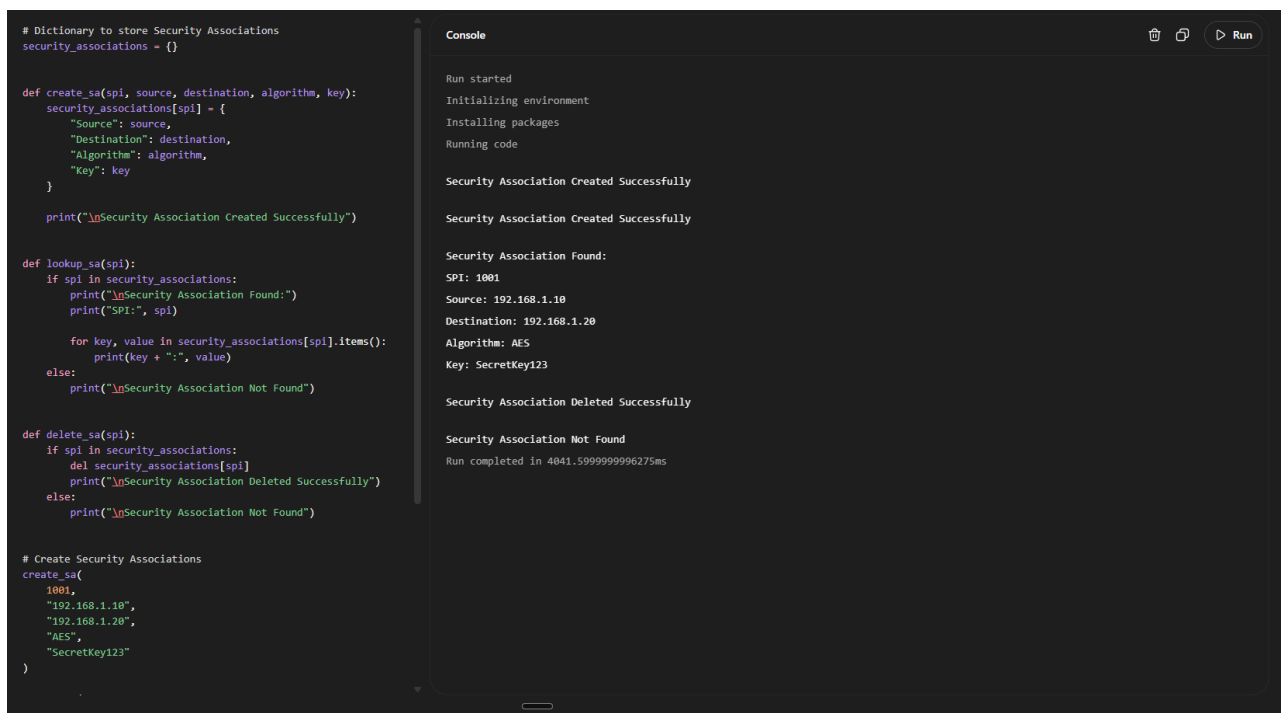
Result:

Thus, the Encapsulating Security Payload mechanism was implemented successfully

3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.

Aim:

To create, look up, and delete Security Associations used in IPSec communication.



```
# Dictionary to store Security Associations
security_associations = {}

def create_sa(spi, source, destination, algorithm, key):
    security_associations[spi] = {
        "Source": source,
        "Destination": destination,
        "Algorithm": algorithm,
        "Key": key
    }
    print("\nSecurity Association Created Successfully")

def lookup_sa(spi):
    if spi in security_associations:
        print("\nSecurity Association Found:")
        print("SPI:", spi)
        for key, value in security_associations[spi].items():
            print(key + ":", value)
    else:
        print("\nSecurity Association Not Found")

def delete_sa(spi):
    if spi in security_associations:
        del security_associations[spi]
        print("\nSecurity Association Deleted Successfully")
    else:
        print("\nSecurity Association Not Found")

# Create Security Associations
create_sa(
    1001,
    "192.168.1.10",
    "192.168.1.20",
    "AES",
    "SecretKey123"
)
```

Console

```
Run started
Initializing environment
Installing packages
Running code

Security Association Created Successfully

Security Association Created Successfully

Security Association Found:
SPI: 1001
Source: 192.168.1.10
Destination: 192.168.1.20
Algorithm: AES
Key: SecretKey123

Security Association Deleted Successfully

Security Association Not Found
Run completed in 4041.5999999996275ms
```

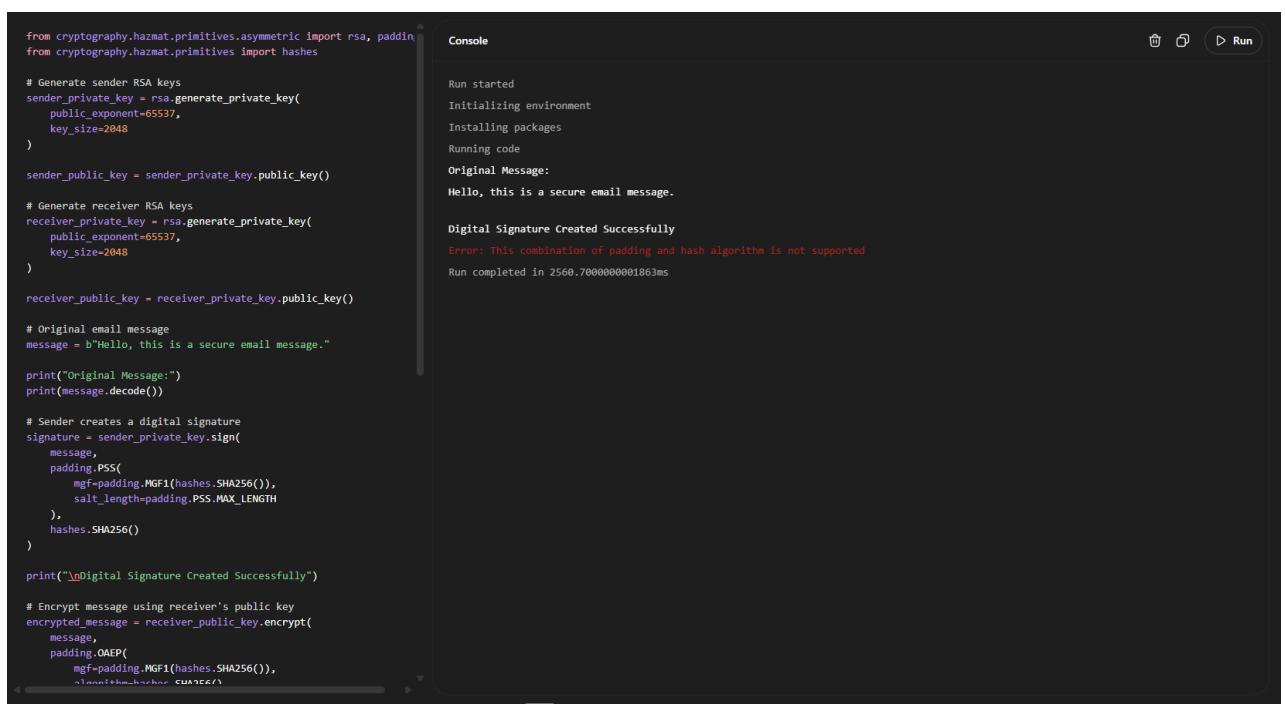
Result:

Thus, Security Association creation, lookup, and deletion operations were successfully implemented using a dictionary-based IPSec SA database.

4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.

Aim:

To implement a simplified PGP system using RSA encryption and digital signatures for secure email communication.



```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generate sender RSA keys
sender_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

sender_public_key = sender_private_key.public_key()

# Generate receiver RSA keys
receiver_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

receiver_public_key = receiver_private_key.public_key()

# Original email message
message = b"Hello, this is a secure email message."

print("Original Message:")
print(message.decode())

# Sender creates a digital signature
signature = sender_private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nDigital Signature Created Successfully")

# Encrypt message using receiver's public key
encrypted_message = receiver_public_key.encrypt(
    message,
    padding.OAEP(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.OAEP.DEFAULT_SALT_LENGTH
    )
)
```

Console

Run started
Initializing environment
Installing packages
Running code

Original Message:
Hello, this is a secure email message.

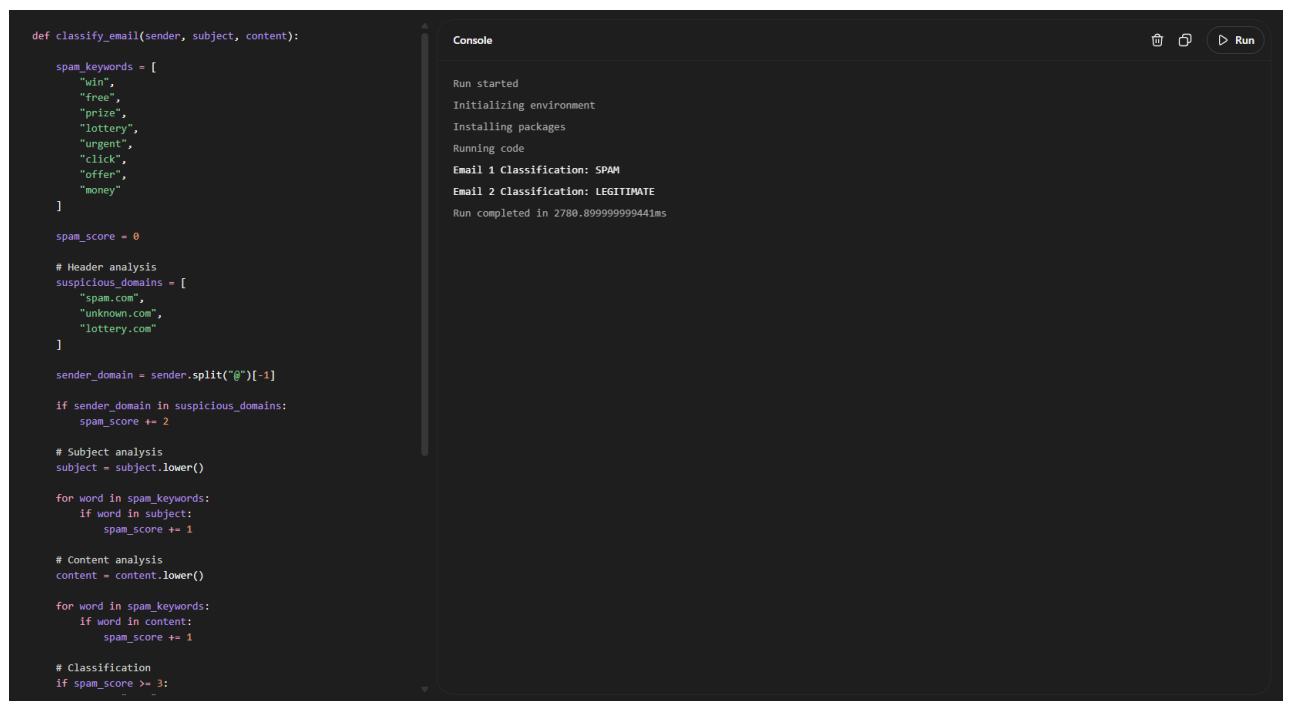
Digital Signature Created Successfully
Error: This combination of padding and hash algorithm is not supported
Run completed in 2560.7000000001863ms

Result:

Thus, a simplified PGP system was successfully implemented using RSA encryption for confidentiality and digital signatures for authentication and integrity.

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

Aim: To classify emails as spam or legitimate using header analysis and content-based filtering techniques.



```
def classify_email(sender, subject, content):  
    spam_keywords = [  
        "win",  
        "free",  
        "prize",  
        "lottery",  
        "urgent",  
        "click",  
        "offer",  
        "money"  
    ]  
  
    spam_score = 0  
  
    # Header analysis  
    suspicious_domains = [  
        "spam.com",  
        "unknown.com",  
        "lottery.com"  
    ]  
  
    sender_domain = sender.split("@")[-1]  
  
    if sender_domain in suspicious_domains:  
        spam_score += 2  
  
    # Subject analysis  
    subject = subject.lower()  
  
    for word in spam_keywords:  
        if word in subject:  
            spam_score += 1  
  
    # Content analysis  
    content = content.lower()  
  
    for word in spam_keywords:  
        if word in content:  
            spam_score += 1  
  
    # Classification  
    if spam_score >= 3:  
        return "SPAM"  
    else:  
        return "LEGITIMATE"
```

Console

```
Run started  
Initializing environment  
Installing packages  
Running code  
Email 1 Classification: SPAM  
Email 2 Classification: LEGITIMATE  
Run completed in 2780.899999999441ms
```

Result:

Thus, the email classification system was successfully implemented using sender header analysis and content-based spam keyword filtering.

